```
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from app.database import get_db
from app import models

SECRET_KEY = "proxypharma_secret_key_2024"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 1440

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")


def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)


def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    if "sub" in to_encode:
        to_encode["sub"] = str(to_encode["sub"])
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)


def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
):
    err = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Token invalide",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
```

```
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")
        if user_id is None:
            raise err
    except JWTError:
        raise err
    user = db.query(models.User).filter(
        models.User.id == int(user_id)
    ).first()
    if user is None:
        raise err
    return user


def require_admin(
    current_user: models.User = Depends(get_current_user),
):
    if current_user.role != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Admin only",
        )
    return current_user
```